



TECHNICAL REPORT: DATA MANAGEMENT AND SCD TYPE 2 IMPLEMENTATION IN APACHE HIVE

Hive Assignment



MAY 12, 2026

Fadi Al-Ariqi

1. Introduction

This project demonstrates the process of managing customer data within Apache Hive. The primary objectives were to handle complex delimiters, compare Internal and External table behaviors, and implement a Slowly Changing Dimension Type 2 (SCD2) strategy to track historical changes without using ACID transactional tables.

2. Handling the Delimiter Challenge

The Problem: The Address column in the source CSV files contains commas (e.g., "123 Street, City"). Standard Hive delimiters fail here as they split the address into multiple incorrect columns. The Solution: Instead of the default *LazySimpleSerDe*, I utilized the *OpenCSVSerde*. This allowed me to define a *quoteChar*, ensuring that any comma inside double quotes is treated as part of the data, not a separator.

Table Schema:

```
CREATE TABLE cust_internal (  
    CustomerID INT, Name STRING, Email STRING, Phone_Number STRING,  
    Address STRING, JOIN_Date STRING, Start_Date STRING, End_Date  
    STRING, Is_Current INT  
)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES ("separatorChar" = ",", "quoteChar" = "\"")  
TBLPROPERTIES ("skip.header.line.count"="1");
```

3. Managed (Internal) vs. External Tables

A practical experiment was conducted by dropping both table types to observe data persistence:

- Managed Table (Internal): Dropping *cust_internal* resulted in Hive deleting both the Metadata (table definition) and the Actual Data from the HDFS warehouse directory.
- External Table: Dropping *cust_external* only removed the Metadata. The physical data files remained safely stored in the HDFS location, proving that External tables are ideal for shared or critical data.

4. SCD Type 2 Implementation (The Workaround)

The Challenge: The current Hive environment does not support *UPDATE* or *DELETE* statements (Non-Transactional). The Workaround: I implemented a *Union-Based Merge Strategy*. This involves creating a new table by combining three distinct datasets:

- 1- Unchanged Records: Existing records that had no corresponding ID in the update file.
- 2- Historical Records (Deactivation): Existing records that matched the update file; these were updated to *Is_Current* = '0' with an *End_Date* set to the current date.
- 3- New Active Records (Insertion): The updated data from the source file, inserted with *Is_Current* = '1' and a new *Start_Date*.

The SCD2 Query:

```
CREATE TABLE customer_scd2_final AS
SELECT
    CAST(old.CustomerID AS STRING), old.Name, old.Email,
    old.Phone_Number, old.Address,
    old.JOIN_Date, old.Start_Date, old.End_Date,
    CAST(old.Is_Current AS STRING) as Is_Current
FROM cust_internal old
LEFT JOIN cust_updates upd ON old.CustomerID = upd.CustomerID
WHERE upd.CustomerID IS NULL
UNION ALL
SELECT
    CAST(old.CustomerID AS STRING), old.Name, old.Email,
    old.Phone_Number, old.Address,
    old.JOIN_Date, old.Start_Date,
    '2026-05-12' as End_Date,
    '0' as Is_Current
FROM cust_internal old
JOIN cust_updates upd ON old.CustomerID = upd.CustomerID
WHERE old.Is_Current = 1
UNION ALL
SELECT
    CAST(upd.CustomerID AS STRING), upd.Name, upd.Email,
    upd.Phone_Number, upd.Address,
    upd.JOIN_Date,
    '2026-05-12' as Start_Date,
    CAST(NULL AS STRING) as End_Date,
    '1' as Is_Current
FROM cust_updates upd;
```

5. Verification and Results

The implementation was validated by querying the customer_scd2_final table. As seen in the output for Customer Larry Hess (ID: 12364), the system successfully tracked the history:

- Row 19 (Expired): Shows the old email/data with Is_Current = 0 and a defined End_Date.
- Row 18 (Active): Shows the new updated record with Is_Current = 1 and a new Start_Date.

CustomerID	Name	Start_Date	End_Date	Is_Current
12364	Larry Hess	2/11/2025	2026-05-12	0
12364	Larry Hess	2026-05-12	NULL	1

```
PS C:\Users\Al-Ariqi Fadi\Documents\docker\Apache Tools\apache-tools-setup> docker compose exec -it itvdelab /bin/bash
hive> SELECT * FROM customer_scd2_final LIMIT 20;
OK
12346 Debra Wells michaelkerr@example.net 3317035462 USNW Olsen FPO AA 43254 2025-02-10 2026-05-12 NULL 1 2026-05
12347 Michael Grimes rchandler@example.net 243.521.3090 36923 Bowers Gateway Suite 027 New Kristi, MP 44312 2025-03-14 2026-05
-12 NULL 1
12348 Jennifer Cuevas brandonbrown@example.com +1-863-584-5200x273 095 Stevenson Lodge New Brettville, LA 67557 2025-01-03 2
026-05-12 NULL 1
12349 Anthony Harvey dgonzales@example.net 771-305-3764x583 549 Taylor Mews Baileyburgh, NJ 59083 2025-04-01 2026-05-12 N
ULL 1
12350 Thomas Martin longpatricia@example.net 558-648-9504x3880 294 Patricia Light Suite 377 Marymouth, GA 49741 2025-02
-07 2026-05-12 NULL 1
12352 James Bush dennis40@example.net +1-650-456-4394x097 44332 Janet Cliffs Apt. 178 Johnsonfort, WA 55056 2025-03-02 2
026-05-12 NULL 1
12353 William Greene floreschristina@example.net 001-609-772-9593x4809 5563 Brian Drives New Charlotteland, NY 81878 2025-03-15 2
026-05-12 NULL 1
12354 Emily Tanner MD anna45@example.com (886)301-5618 020 Willie Springs Apt. 266 Sueberg, GA 65464 2025-01-29 2026-05-12 N
ULL 1
12355 James Acevedo josephbrown@example.com 339-695-7693 1347 Taylor Station Suite 362 South Scottport, NH 32618 2025-02-24 2026-05
-12 NULL 1
12356 Melinda Kennedy mary87@example.net +1-420-680-5658x92614 527 Hunter Lane Apt. 717 Phillipstown, MS 10123 2025-01-19 2026-05
-12 NULL 1
12357 Sherri Mora christophercook@example.com 458.978.5946 16245 Jason Rest Suite 741 Nortonmouth, MA 47922 2025-01-19 2
026-05-12 NULL 1
12358 Lucas Wood george41@example.com (579)527-3515x6724 922 Bell Valleys Apt. 515 Hansenmouth, DC 09410 2025-03-27 2026-05
-12 NULL 1
12359 Alexandra Smith hensonjames@example.com 403.399.4781 4707 Mendoza Shoal Suite 365 New Gregory, IL 47589 2025-03-06 2026-05
-12 NULL 1
12360 Sean Anderson powens@example.org 839-365-2193 04795 Hays Forges Suite 301 Millerburgh, NM 47791 2025-02-01 2026-05
-12 NULL 1
12361 Jeanette Stark wfrancis@example.org 001-263-530-4234x4700 4209 Guzman Burg South Cynthia, KS 85592 2025-02-23 2026-05
-12 NULL 1
12362 Albert Bishop baileynicole@example.com 315-873-7701x282 617 Brian Garden Simpsonhaven, RI 49804 2025-03-30 2026-05
-12 NULL 1
12363 Haley Sullivan elizabethhardy@example.net 994-617-2939x962 664 Michael Turnpike Suite 930 Port Aprilmouth, PR 61100 2
025-03-08 2026-05-12 NULL 1
12364 Larry Hess meyerjennifer@example.com 001-626-971-5766 556 Schmidt Gateway Apt. 111 Kaneton, NM 59573 2025-02-11 2
026-05-12 NULL 1
12364 Larry Hess upd_meyerjennifer@example.com 001-626-971-5766 556 Schmidt Gateway Apt. 111 Kaneton, NM 59573 2/11/2025 4
/1/2025 2026-05-12 0
12365 Nancy Shaw MD greed@example.net (561)740-1250 Unit 3498 Box 9802 DPO AE 01301 2025-02-16 2026-05-12 NULL 1
```

0 139 Go Live

6. Conclusion

By leveraging advanced SerDe properties and a Union-based logic, I successfully bypassed the limitations of non-transactional Hive tables. This ensures that the data warehouse can maintain a full history of customer changes, which is vital for accurate time-series analysis and reporting.